# SWC3

## Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



# 3. C Grundlagen

# 3.1 C Grundlagen: Benutzerdefinierte Datentypen

# Aufzählungen (*enum*)

- Erzeugen von benutzerdefinierten Bezeichnern für numerische Werte

- neue Bezeichner von `color_t` bekommen passenden Integer-Typ
- Werte beginnen bei Null: `RED=0`, `GREEN=1`, `BLUE=2`

```
enum color_t { RED, GREEN, BLUE };
enum color_t color = RED;
SHOW_TYPE(color);
SHOW_TYPE(GREEN);
```

```
color = 0 (unsigned int)
GREEN = 1 (int)
```

Enumerations - cppreference.com
SHOW_TYPE() und SHOW() in Anhang A – Hilfsfunktionen

- z.B. Bezeichner bekommen Typ `char`
- Wert kann angegeben werden
  - folgende Bezeichner zählen weiter

```
enum ascii_t : char {
  LETTER_A='a', LETTER_B
};
enum ascii_t letter = LETTER_B;
SHOW_TYPE(letter);
SHOW_TYPE(LETTER_A);
```

```
letter = 'b' (char)
LETTER_A = 'a' (char)
```

- Typname ist »`enum ascii_t`«
  - nicht einfach nur »`ascii_t`«
  - eigener Tag-*Namespace* für Aufzählungstypen

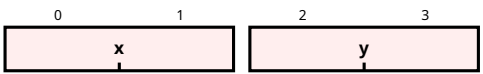
# Verbunddatentypen (*struct*) (i)

- Verbunddatentyp mit mehrere Komponenten
  - eigener Tag-*Namespace* (wie `enum`)
  - kein direkter Vergleich von `struct`s
    - sogar vom gleichen Typ
  - Elementzugriff `p.x` mit Punktnotation
  - Initialisierung mit geschwungenen Klammern
    - entweder Komponenten der Reihe nach
    - oder mit Punktnotation `{.x = 1}`, fehlende Element auf `0` gesetzt

```
struct point_t {
  int16_t x;
  int16_t y;
};
```

```
SHOW(sizeof(struct point_t));
```

```
sizeof(struct point_t) = 4
```




Struct declaration - cppreference.com

```
struct point_t p; // not initialized
struct point_t p0 = {}; // zero initializer
struct point_t p1 = {1, 2};
struct point_t p2 = { .y = 2 };
SHOW_XY(p); SHOW_XY(p0); SHOW_XY(p1); SHOW_XY(p2);
p.x = 123; // member access
SHOW(p.x);
```

```
p = (-258, -258)
p0 = (0, 0)
p1 = (1, 2)
p2 = (0, 2)
p.x = 123
```

```
struct point_t p0 = {};
struct point_t p1 = {1, 2};
p0 = p1; // direct assignment
SHOW_XY(p0);
p0 = (struct point_t){3, 4}; // compound literal
SHOW_XY(p0);
```

```
p0 = (1, 2)
p0 = (3, 4)
```

# Verbunddatentypen (*struct*) (ii)

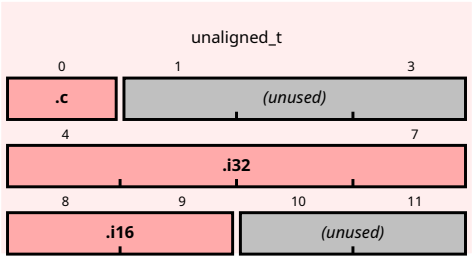
- von Compiler an Architektur ausgerichtet (*aligned*)

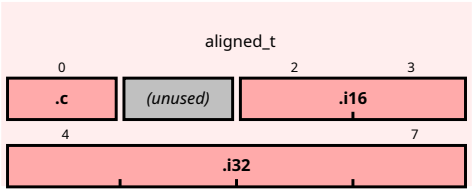
```
struct unaligned_t {
  char c;      //   1 byte
  int32_t i32; // + 4 byte
  int16_t i16; // + 2 byte
};             // = 7 byte?
SHOW(sizeof(struct unaligned_t));
```

```
sizeof(struct unaligned_t) = 12
```

```
struct aligned_t {
  char c;      //   1 byte
  int16_t i16; // + 2 byte
  int32_t i32; // + 4 byte
};             // = 7 byte?
SHOW(sizeof(struct aligned_t));
```

```
sizeof(struct aligned_t) = 8
```



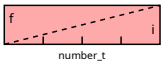




Structures in C: From Basics to Memory Alignment

# Vereinigungstypen (*union*)

- Vergabe von verschiedenen Typen für den gleichen Speicher
  - `i` und `f` belegen gleichen Speicher

```
union number_t {
  float f;
  uint32_t i;
};
union number_t n = {};
                  SHOW_NUMBER(n);
n.f = 1;          SHOW_NUMBER(n);
n.i += (1<<21); SHOW_NUMBER(n);
SHOW(sizeof(float));
SHOW(sizeof(uint32_t));
SHOW(sizeof(union number_t));
```




Union declaration - cppreference.com

```
union float_t {
  float value;
  struct {
    unsigned mant : 23;
    unsigned exp  : 8;
    unsigned sign : 1;
  };
};
union float_t f = {};
                   SHOW_FLOAT(f);
f.value = -1.5; SHOW_FLOAT(f);
f.value =  3;   SHOW_FLOAT(f);
f.mant =   0;   SHOW_FLOAT(f);
```

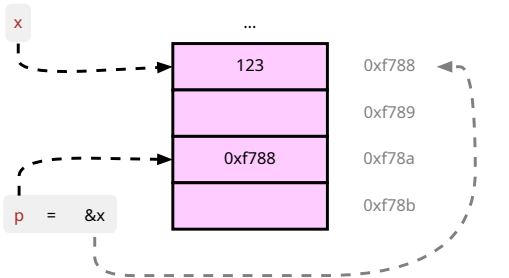
```
n = {.f= 0.00, .i=0b00000000000000000000000000000000}
n = {.f= 1.00, .i=0b00111111100000000000000000000000}
n = {.f= 1.25, .i=0b00111111101000000000000000000000}
sizeof(float) = 4
sizeof(uint32_t) = 4
sizeof(union number_t) = 4
```

```
f = {.value= 0.00 .sign=+ .exp=0x00 .mant=0x000000}
f = {.value=-1.50 .sign=- .exp=0x7f .mant=0x400000}
f = {.value= 3.00 .sign=+ .exp=0x80 .mant=0x400000}
f = {.value= 2.00 .sign=+ .exp=0x80 .mant=0x000000}
```

# Zeiger (*pointer*)

- (jeder!) Variablenname steht für Adresse des Inhalts
  - zur Laufzeit verwendet CPU statt Variablennamen nur Adresse
- Deklaration von Zeigertyp mit Hilfe von Sternchen (*asterisk*) »`*`«
- Zeiger enthalten Adresse als Wert, z.B. die Adresse einer anderen Variablen
- Die Adresse einer Variablen erhält man mit dem Adressoperator »&«
- Den Wert auf den gezeigt wird erhält man mit Dereferenzierungsoperator »`*`«

```
int x = 123; // [0xf788] = 123;
int* p = &x; // [0xf78a] = 0xf788;
```

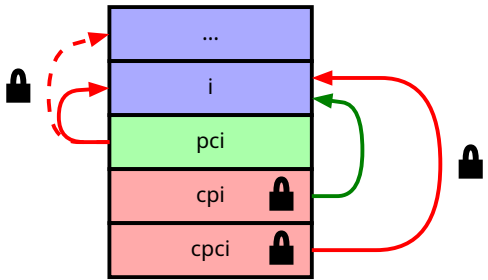


# Konstante Zeiger

bei Zeiger und andere komplexere Datentypen kann mehr als ein Teil konstant sein

- Leserichtung von *rechts nach links*, z.B. für `int i`

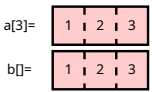
- `const int * pci = &i`
  - `pci` ist
  - (veränderbarer) Zeiger
  - auf `int`-Konstante
- `int * const cpi = &i`
  - `cpi` ist
  - konstanter Zeiger
  - auf (veränderbaren) `int`
- `const int * const cpci = &i`
  - `cpci` ist
  - konstanter Zeiger
  - auf `int` Konstante



# Felder (*arrays*)

- zusammenhängende Folge von Werten gleichen Typs

- mit fixer Länger
- die zur Übersetzungszeit bekannt ist
- Definition mit eckigen Klammern (*brackets*) »`[]`«
- Zugriff mit Index-Operator `a[0]`
- Start bei Index Null




```
char a[3] = {1, 2, 3};
SHOW(sizeof(a));

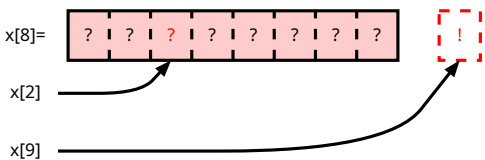
char b[] = {1, 2, 3}; // auto size
SHOW(sizeof(b));
```

```
sizeof(a) = 3
sizeof(b) = 3
```

Array declaration - cppreference.com

# Felder sind *“roh”*

- keine automatische Initialisierung
- keine Bereichsprüfung (*bounds check*)




- erlaubt höhere Performance
- birgt Risiken durch geringe Speichersicherheit

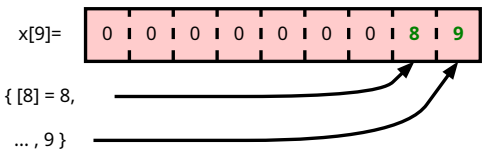
```c
int x[8]; // XXX: not initialized
SHOW(x[2]);
SHOW(x[9]); // XXX: no bounds check
```

```
x[2] = 1753324224
x[9] = 32764
```

Ausgabe ohne Sanitizer

# Felder: Initialisierung

- Initialisieren und Zugriff mit Indexoperator, z.B. `c[0]`
- sobald Initialisierer angegeben »`{}`«
  - auch andere Indizes 0-initialisiert




- Array-Größe in Bytes kann (manchmal!) mit `sizeof` abgefragt werden
  - z.B. wenn die Größe zur Übersetzungszeit bereits feststeht

```
int c[10] = {};
SHOW(c[0]);
SHOW(c[9]);
```

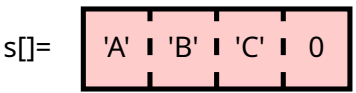
```
c[0] = 0
c[9] = 0
```

```
int c[] = { [8] = 8, 9 };
SHOW(c[0]);
SHOW(c[8]);
SHOW(c[9]);
SHOW(sizeof(c)); // 10*sizeof(int)
SHOW(sizeof(c)/sizeof(c[0]));
```

```
c[0] = 0
c[8] = 8
c[9] = 9
sizeof(c) = 40
sizeof(c)/sizeof(c[0]) = 10
```

# Felder und C-Zeichenketten

- Felder sind Basis für C-Zeichenketten (*C strings*)
  - kennen eigene Länge nicht
  - Terminatorzeichen am Ende
    - ASCII Code `0`
- sehr häufige Fehlerquelle
  - z.B. Pufferüberlauf (*buffer overflow*)




```c
char s[] = "ABC"; // + '\0'
SHOW(sizeof(s));
SHOW(s[0]);
SHOW((int)s[3]);
SHOW(s);
```

```
sizeof(s) = 4
s[0] = 'A'
(int)s[3] = 0
s = "ABC"
```

# Felder mit variabler Länge

- Seit C99 auch Felder mit variabler Länge möglich (*variable-length arrays*, VLA)
  - Compiler erzeugt Code zum Initialisieren und Freigeben automatisch

```
srand(time(nullptr)); // initialize RNG
size_t size = rand() % 20; // random size
SHOW(size);
char a[size];
SHOW(sizeof(a));
```

```
size = 7
sizeof(a) = 7
```

was ist das problem damit?

# Felder zerfallen bei Parameterübergabe (i)

- Verfügbarkeit von VLAs irreführend, häufig ist Länge nicht bekannt
- bei Parameterübergabe zerfällt Array zu Zeiger (*decay*)
  - in Funktion somit nicht Länge von Array sondern Größe von `int`-Zeiger

```c
/// NOTE: incorrect implementation
void show_length(int a[static 1]) {
  printf("--- inside function: ---\n");
  SHOW(sizeof(a)); // XXX: size of int pointer
  SHOW(sizeof(a[0])); // size of int
  SHOW(sizeof(a) / sizeof(a[0]));
}

int main() {
  int a[] = {1};
  SHOW(sizeof(a));
  SHOW(sizeof(a[0]));
  size_t len = sizeof(a)/sizeof(a[0]);
  SHOW(len);

  show_length(a);
  show_length(nullptr);
}
```

```
sizeof(a) = 4
sizeof(a[0]) = 4
len = 1
--- inside function: ---
sizeof(a) = 8
sizeof(a[0]) = 4
sizeof(a) / sizeof(a[0]) = 2
--- inside function: ---
sizeof(a) = 8
sizeof(a[0]) = 4
sizeof(a) / sizeof(a[0]) = 2
```

```c
SHOW(sizeof(int*));
SHOW(sizeof(int));
```

```
sizeof(int*) = 8
sizeof(int) = 4
```

# Felder zerfallen bei Parameterübergabe (ii)

- korrekte Implementierung nur mit zusätzlichem Parameter möglich

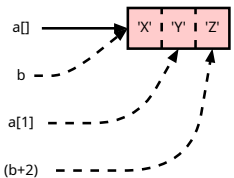
```c
/// correct implementation
void show_length(size_t len, int a[static len]) {
  printf("--- inside function: ---\n");
  SHOW(len);
}

int main() {
  int a[] = {1};
  SHOW(sizeof(a));
  SHOW(sizeof(a) / sizeof(a[0]));
  show_length(sizeof(a) / sizeof(a[0]), a);
  show_length(0, nullptr);
}
```

```
sizeof(a) = 4
sizeof(a) / sizeof(a[0]) = 1
--- inside function: ---
len = 1
--- inside function: ---
len = 0
```

# Felder sind Zeiger

- meist sind Felder einfach Zeiger auf erstes Element
  - mit Typ-Information (also Elementgröße)
  - aber ohne Information über die Gesamtlänge des Feldes oder die Anzahl der Elemente
- Felder und Zeiger sind meist austauschbar
- Zeiger-Arithmetik is äquivalent zu Feldindexierung




```
char a[] = {'X', 'Y', 'Z'};
char* b = a; // copy address
```

```
SHOW(a[0]);
SHOW(*b);
```

```
a[0] = 'X'
*b = 'X'
```

```
SHOW(a[1]);
SHOW(*(b+1));
```

```
a[1] = 'Y'
*(b+1) = 'Y'
```

```
SHOW(*(a+2));
SHOW(b[2]);
```

```
*(a+2) = 'Z'
b[2] = 'Z'
```

```
b[2] = 'C';
SHOW(a[2]);
```

```
a[2] = 'C'
```

# Benutzerdefinierte Typ-Namen (*typedef*)



- Vergabe von Pseudonymen für (möglicherweise komplexe) Typen

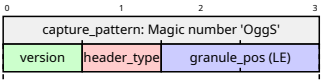
```
enum color_t { RED, GREEN, BLUE };
typedef enum color_t color_array_t[];
color_array_t color = { BLUE, GREEN, RED };
```

- oft in Kombination mit (anonymen) `enum` oder `struct`

```
typedef struct /* no struct tag name */ {
  double x;
  double y;
} point_t;
point_t p1 = {0.1, 0.2};
```

```
typedef enum /* no enum tag name */ {RED, GREEN, BLUE} color_t;
color_t color = RED;
```

# Beispiel Ogg-Header (gekürzt)




```c
typedef enum : uint8_t {
  CONTINUE = 1 << 0,
  BEGIN    = 1 << 1,
  END      = 1 << 2,
} flags_t;

typedef struct {
  char magic[4];
  uint8_t version;
  flags_t header_type;
  uint16_t granule_pos;
} header_t;

typedef union {
  header_t fields;
  uint8_t bytes[8];
} packet_t;
```

```c
int main() {
  packet_t ogg_header = {
    .fields = {
      {'O', 'g', 'g', 'S'},
      1, CONTINUE | END, 0xABCD
    }
  };

  SHOW(sizeof(ogg_header));
  SHOW(sizeof(header_t));
  SHOW(sizeof(packet_t));
  SHOW(ogg_header.fields.header_type);
  SHOW_HEX_CHARS(ogg_header.bytes);
}
```

```
sizeof(ogg_header) = 8
sizeof(header_t) = 8
sizeof(packet_t) = 8
ogg_header.fields.header_type = 5
 0: 0x4f 'O' |  1: 0x67 'g' |  2: 0x67 'g' |  3: 0x53 'S' |
 4: 0x01  ?  |  5: 0x05  ?  |  6: 0xcd  ?  |  7: 0xab  ?  |
```

RFC 3533 - Ogg Encapsulation Format v0

# 3.2 C Grundlagen: Ausdrücke

# Ausdrücke (*expressions*) & Operatoren – Überblick

Folge von Operatoren und Operanden, erzeugt Ergebnis (Wert)

- Arten von Ausdrücken
  - primäre Ausdrücke
    - Literale, z.B. `'a'`, `"abc"`, `123`
    - Variablen, z.B. `i`, `count`
  - Arithmetische und logische Ausdrücke
    - z.B. `a + b`
  - Vergleiche
    - z.B. `a < b`
  - Elementzugriff
    - z.B. `a[b]`
  - Zuweisungen
    - z.B. `a = b`
  - Andere
    - z.B. `sizeof(a)`

Expressions - cppreference.com

# Ausdrücke – Arithmetik, Logik, Zuweisungen, Vergleiche

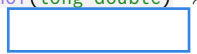
- Vorzeichen (`+a`, `-a`)
- Rechnen
  - `a * b`, `a + b`, `a - b`, `a * b`, `a / b`, `a % b`
  - Modulo-Operation / Rest der Division (`%`) nur für ganze Zahlen
- Inkrement und Dekrement
  - `a++`, `++a`, `a--`, `--a`
- Bit-Operationen
  - Negation `~a`
  - AND `a & b`, OR `a | b`, XOR, `a ^ b`
  - Left-Shift `a << b`, Right-Shift `a >> b`
- Logische Operationen
  - Verneinung `!a`, AND `a && b`, OR `a || b`
  - Auswertung im Kurzschlussverfahren

```
true || printf("logic"); // --> no output
true | printf("bitwise"); // --> "bitwise"
```

- Einfache Zuweisung `a = b`
  - liefert zugewiesenen Wert
  - kann z.B. verkettet werden `a = b = c = d`
- Arithmetische Zuweisung
  - `a -= b`, `a *= b`, `a /= b`, `a %= b`, `a |= b`, `a ^= b`, `a <<= b`, `a >>= b`
  - verkürzte Schreibweise, kombiniert Operation und Zuweisung
    - z.B. `a += b` → `a = a + b`
- Vergleiche
  - `a == b`, `a != b`
  - `a < b`, `a <= b`, `a >= b`, `a > b`

# Ausdrücke – Weitere Operatoren

- Funktionsaufruf `printf()`
- Elementzugriffe
  - Indexoperator für Feld `a[b]`
  - Punktoperator für Verbund (*struct*) `a.b`
  - Dereferenzierung von Zeiger `*a`
  - Adresse von Bezeichner `&a`
  - Kombination von Dereferenzierung und Verbundzugriff `a->b` ⇔ `(*a).b`
- `sizeof(a)`
  - Größe eines Typs oder Ausdrucks in Byte
  - z.B. `sizeof(int_fast32_t)` → 8
- `alignof(a)`
  - Alignment eines Typs oder Ausdrucks
  - z.B. `alignof(char)` → 1, `alignof(long double)` → 16



- Sequenz-Operator `a, b`
  - führt beide Ausdrücke aus
  - liefert Ergebnis des zweiten

```
int a;
(a = 123, 456) // --> 456
a // --> 123
```

  - selten benutzt
    - z.B. Intitialisierung von `for`-Schleife
- Typumwandlung (*cast*) `(type)a`
- Ternärer Operator `a ? b : c`
  - `if` als Ausdruck
  - falls `a` wahr ist liefert `b`
  - sonst `c`

```
10 > 5 ? "big" : "small" // --> "big"
```

# Reihenfolge der Operator-Auswertung (*precedence*)

Operatoren haben eine definierte Auswertungsreihenfolge

- Priorität 1 ⟶ (von Links nach Rechts)
  - suffix Inkrement/Dekrement `a++`, `a--`
  - Funktionsaufruf, Feldzugriff `a()`, `a[b]`
  - Verbundliterale `(point_t){1, 2}`
- Priorität 2 ⟵ (von Rechts nach Links)
  - Prefix Inkrement/Dekrement `++a`, `--a`
  - Vorzeichen & Verneinung `+a`, `-a`, `!a`, `~a`
  - Deref- & Adressoperator `*a`, `&a`
  - `sizeof(a)`, `alignof(a)`
- Priorität 3, 4 ⟶
  - Multiplikation & Division `a*b`, `a/b`
  - Addition & Subtraktion `a+b`, `a-b`
- Priorität 5 ⟶
  - Bitshift `a>>b`, `a<<b`
- Priorität 6, 7 ⟶
  - Ungleichheit `a<b`, `a<=b`, `a>=b`, `a>b`
  - Gleicheit `a==b`, `a!=b`
- Priorität 8, 9, 10 ⟶
  - Bitoperationen `a & b`, `a^b`, `a|b`
- Priorität 11, 12 ⟶
  - Logische Operationen `a && b`, `a || b`
- Priorität 13 ⟵
  - Ternärer Operator `a ? b : c`
- Priorität 14 ⟵
  - Zuweisungsoperatoren `a = b`, `a += b`, …
- Priorität 15 ⟶
  - Sequenzoperator `a, b`

**Tipp**

Im Zweifelsfall Klammern verwenden

# Reihenfolge der Auswertung von Operanden

Operanden auf gleicher Ebene haben keine definierte Auswertungsreihenfolge

- außer bei Kurzschlussevaluierung

```
printf("a") && printf("b"); // --> ab
printf("a") + printf("b"); // XXX: undefined
//    --> ab
// OR --> ba
a[i] = a[++i]; // XXX: undefined
//    --> a[i] = a[i+1]; i = i+1;
// OR --> a[i+1] = a[i]; i = i+1;
```

Seiteneffekte in Ausdrücken generell vermeiden, besser

```
a[i] = a[i+1];
++i;
```

- Optimierung dem Compiler überlassen[1]

[1] beide Versionen erzeugen den gleichen Maschinencode https://godbolt.org/z/e3zzx63Y8

# Arithmetische Ausdrücke mit schmalen Datentypen

- sogenannte schmalen Datentypen (*narrow types*)
  - `bool`, `unsigned char`, `unsigned short`
  - `char`, `signed char`, `short`
- werden vor der Berechnung befördert (*promoted*)
  - das Ergebnis ist meist vom Typ `int`

```
short x = 1;
short y = 2;
SHOW_TYPE(x);
SHOW_TYPE(y);
SHOW_TYPE(x + y);
```

```
x = 1 (short int)
y = 2 (short int)
x + y = 3 (int)
```

```
bool a = false;
bool b = true;
SHOW_TYPE(a);
SHOW_TYPE(b);
SHOW_TYPE(a + b);
SHOW_TYPE(a && b);
```

```
a = 0 (bool)
b = 1 (bool)
a + b = 1 (int)
a && b = 0 (int)
```

# Arithmetische Ausdrücke mit unterschiedlichen Datentypen (i)

```
int i = 1;
long l = 2;
long long ll = 3;
SHOW_TYPE(l+i);
SHOW_TYPE(i+ll);
i = ll + l; // NOTE: narrowing conversion
SHOW_TYPE(i);
```

```
l+i = 3 (long int)
i+ll = 4 (long long int)
i = 5 (int)
```

```
float f = 1;
double d = 2;
long double ld = 3;
SHOW_TYPE(f+d);
SHOW_TYPE(f+ld);
f = d + ld; // NOTE: narrowing conversion
SHOW_TYPE(f);
SHOW_TYPE(f+f);
```

```
f+d = 3 (double)
f+ld = 4 (long double)
f = 5 (float)
f+f = 10 (float)
```

- Compilerwarnung für implizite Konvertierungen, z.B. GCC mit `-Wconversion`

# Arithmetische Ausdrücke mit unterschiedlichen Datentypen (ii)

```c
float f = 1;
long int i = LONG_MAX;
SHOW_TYPE(i);
SHOW_TYPE(f+i); // NOTE: narrowing conversion (long int -> float)
```

```
i = 9223372036854775807 (long int)
f+i = 9.22337e+18 (float)
```

- implizite Typumwandlung generell vermeiden

# 3.3 C Grundlagen: Anweisungen & Kontrollfluss

# Einfache Anweisungen

- leere Anweisung »`;`«
- Ausdrucksanweisung »`expression;`«

```
a + b; // no effect
a = b;
f(a);
```

- Blockanweisung durch Klammerung »`{ /* block */ }`«
  - kann eigene Deklarationen und Definitionen enthalten
  - kann beliebige Anweisungen enthalten
  - kein Semikolon am Ende

```
int a = 1;
{
  int b = a + 1;
}
{
  int c = a + 2;
}
```

# Verzweigungen

- Binäre Verzweigung mit `if`
  - wenn Bedingung (*condition*) ungleich `0` (=false) wird Konsequenz ausgeführt
  - ansonsten die Alternative (falls mit `else` angegeben)

```
if (condition_expr)
  consequence_statement
else
  alternative_statement
```

- Mehrfachverzweigung mit `switch` und `case`
  - Entscheidungsausdruck muss ganzzahlig sein (ein `int`-Typ)

```
switch(expr) {
  case 1: statement1; ...; // fall-through to case 2
  case 2: statement2; ...; break;
    ...
  default: default_statement;
}
```

# Schleifen

- Abweis- und Durchlaufschleife `while` und `do ... while`
  - solange Bedingung ungleich `0` ist
  - wird die Anweisung (oder Block) ausgeführt

```
while (expr) statement;
while (expr) { statement1; statement2; ... }
do statement while(expr);
do { statement1; statement2; ...} while(expr);
```

- Zählschleife `for`
  - `init_expr` wird einmal am Anfang der Schleife ausgeführt
    - Variablen die hier definiert werden überdauern das gesamte if-Statement
  - solange `cond_expr` ungleich `0` ist wird der Ausdruck (oder Block) ausgeführt
  - am Ende jedes Durchlaufs wird `loop_expr` ausgeführt

```
for (init_expr; cond_expr; loop_expr) statement
for (init_expr; cond_expr; loop_expr) {
  statement1; statement2; ...
}
```

  - jeder der drei Ausdrücke kann weggelassen werden, z.B.
    `printf("forever, "); for(;;) printf("and ever, ");`

# Schleifenabbruch

- mit Hilfe von `break` wird eine Schleife sofort abgebrochen
- mit Hilfe von `continue` wird sofort zur nächsten Iteration gesprungen

```c
for (size_t i = 0; i<10; i++) {
  if (i < 5) continue;
  if (i > 7) break;
  printf("i = %ld\n", i);
}
```

```
i = 5
i = 6
i = 7
```

# Sprünge mit `goto`

- mit Hilfe von `goto` kann innerhalb einer Funktion gesprungen werden

```
goto hello;

world: printf("World!");
       goto end;

hello: printf("Hello, ");
       goto world;

end:
```

```
Hello, World!
```

- in der Praxis kaum mehr verwendet bzw. notwendig

# Sprünge mit `longjmp()`

- Mit Hilfe von `longjmp()` kann zwischen Funktionen gesprungen werden

```c
#include <setjmp.h>
jmp_buf target;

[[noreturn]] void f(size_t depth) {
  printf("recursion %ld\n", depth);
  if (depth > 3) {
    longjmp(target, 1); // (4) jump to saved target
  } //                                          |
  f(depth+1); // (3) infinte recursion          |
} //                                           |
int main() { //                                |
  // (1) mark jump target with setjmp          |
  int result = setjmp(target); // <-----------'
  if (result == 0 ) {
    printf("starting...\n");
    f(0); // (2) call f
  } else {
    printf("result = %d", result);
  }
}
```

```
starting...
recursion 0
recursion 1
recursion 2
recursion 3
recursion 4
result = 1
```

- wird selten direkt verwendet
- erlaubt Implementieren eigener Ablaufsteuerung, z.B.
  - Ausnahmen (*exceptions*)
  - Koroutinen
    - Generatoren
    - kooperatives Multitasking

# 3.4 C Grundlagen: Funktionen

# Deklaration vs. Definition

```c
int max(int a, int b); // modern declaration
int max(int, int); // compatible declaration
max(); /* very old declaration: forbidden in C23 */

int max(int a, int b) {
  return a > b ? a : b;
}
```

normalerweise aufgeteilt auf zwei Dateien

- Header mit Deklarationen `*.h`

```c
// file: max.h

/// return the maximum of a and b
int max(int a, int b);
```

- Quelltext mit Definitionen `*.c`

```c
// file: max.c

int max(int a, int b) {
  return a > b ? a : b;
}
```

# Parameterübergabe (i)

- Parameterübergabe immer per Value

```
int increment(int x) {
  x = x + 1; // copy is incremented (2)
  return x;  //   and returned
}

int main() {
  int x = 0;
  increment(x); // x is copied (1)
  SHOW(x); // x in main is unaffected
  x = increment(x); // x is set to return value
  SHOW(x);
}
```

```
x = 0
x = 1
```

# Parameterübergabe (ii)

- in altem C-Code daher häufig Übergabe per Pointer
  - wenn Referenzsemantik gewünscht wird

```
void increment(int *x) {
  // x -points-to-> counter
  ++(*x); // *x == counter
}

int main() {
  int counter = {};
  increment(&counter); // address of counter is copied
  SHOW(counter);
}
```

```
counter = 1
```

# Parameterübergabe (iii)

- falls Pointer übergeben wird aber keine Änderung gemacht wird → Parameter mit `const` qualifizieren

```
void print(const int *x) { // const => reference not changed
  SHOW(x);
  SHOW(*x);
}
int main() {
  int counter = {};
  print(&counter); // reference is transferred
}
```

```
x = 0x2ef0020
*x = 0
```

- erlaubt bessere Einschätzung beim Aufrufer
- erlaubt Compiler-Optimierungen

# Parameterübergabe (iv)

- Arrays zerfallen (*decay*) zu einfachen Zeigern bei der Parameterübergabe
  - C23 ermöglicht Angabe von Mindestgröße
  - erzeugt aber *nur Warnung*

```c
void array_length(size_t n, int array[static n]) {
  SHOW(sizeof(array)); // --> warning sizeof-array-argument)
}

void not_null(int pointer[static 1]) {
  SHOW(sizeof(pointer)); // --> warning sizeof-array-argument
}

int main() {
  int a[] = {1, 2, 3};
  const size_t len = sizeof(a)/sizeof(a[0]);
  array_length(len, a); // --> OK: size matches
  array_length(10, a);  // --> warning stringop-overflow
  not_null(a);       // --> OK: a is not null
  not_null(nullptr); // --> warning nonnull
}
```

```
sizeof(array) = 8
sizeof(array) = 8
sizeof(pointer) = 8
sizeof(pointer) = 8
```

# Variable Parameteranzahl

<stdarg.h> und »`...`« für *variadische* Funktionen

```c
double sum(int count, ...) {
  double sum = {};
  va_list args; va_start(args, count);
  for (int i = 0; i < count; i++) {
    sum += va_arg(args, double);
  }
  va_end(args);
  return sum;
}

int main() {
  SHOW(sum(0));
  SHOW(sum(2, 1.5, 2.5));
  SHOW(sum(3, 1.1, 2.2, 3.3));
}
```

```
sum(0) = 0
sum(2, 1.5, 2.5) = 4
sum(3, 1.1, 2.2, 3.3) = 6.6
```

```c
void showf(const char fmt[static 1], ...) {
  va_list args;
  for (va_start(args, fmt); *fmt != '\0'; fmt++) {
    switch (*fmt) {
      case 'i': SHOW(va_arg(args, int)); break;
      case 'd': SHOW(va_arg(args, double)); break;
      case 'c': SHOW(va_arg(args, int)); break;
      default: printf("unknown format '%c'", *fmt);
    }
  }
  va_end(args);
}
int main() {
  showf("idcx", 1, 2.2, 'c', "unknown");
}
```

```
va_arg(args, int) = 1
va_arg(args, double) = 2.2
va_arg(args, int) = 99
unknown format 'x'
```

# Rückgabe von Werten

```c
typedef struct { double x; double y; } point_t;
```

- modernes C: direkter Rückgabewert
  - auch bei komplexerem Typ

```c
point_t scale(point_t p, double length) {
  return (point_t){ // return another copy
    .x = p.x * length,
    .y = p.y * length
  };
}
int main() {
  point_t p1 = {1., 1.};
  point_t p2 = scale(p1, 2); // copy p
  SHOW(p2.x);
  SHOW(p2.y);
}
```

```
p2.x = 2
p2.y = 2
```

- älteres C: Rückgabe oft über Zeiger
  - als Referenzparameter
  - Rückgabewert für Fehlercode

```c
int scale(point_t* p_ref, double length) {
     p_ref->x *= length; // modify p in main
     p_ref->y *= length; //   via reference
     return 2; // OK
}
int main() {
  point_t p = {1., 1.};
  if (scale(&p, 2)) { // <-- pass address with `&'
    SHOW(p.x);
    SHOW(p.y);
  }
}
```

```
p.x = 2
p.y = 2
```

# Hauptfunktion (*main*)

`int main()` ist Startpunkt

- Rückgabetyp `int`
  - Rückmeldung an Betriebssystem
    - `== 0` → alles OK
    - `!= 0` → Fehler(code)
- ohne Parameter `int main()`
- mit Kommandozeilenparametern
  - `int argc` *arg. count* → Anzahl Parameter
  - `char *argv[static 1]` *arg. vector* → Werte als Zeichenketten
    - `argv[0]` → (meistens) Programmname

```c
// file: main.c
int main(int argc, char* argv[static argc+1]) {
  printf("list of arguments:\n");
  for (int i = 0; i < argc+1; i++) {
    printf("- argv[%d] = %s\n", i, argv[i]);
  }
  // return EXIT_SUCCESS; (default)
}
```

```
gcc -o main main.c && ./main
```

```
list of arguments:
- argv[0] = ./main
- argv[1] = (null)
```

---

```
./main hello
```

```
list of arguments:
- argv[0] = ./main
- argv[1] = hello
- argv[2] = (null)
```

---

```
./main test 1 2 "3 4"
```

```
list of arguments:
- argv[0] = ./main
- argv[1] = test
- argv[2] = 1
- argv[3] = 2
- argv[4] = 3 4
- argv[5] = (null)
```

# Ausgabefunktion `printf` (i)

```c
#include <stdio.h>

int printf(const char *restrict format, ...)
```

- formatierte Ausgabe
- variable Anzahl von Parametern
- Format-Zeichenkette mit eingebetteten Formatierungshinweisen und Escape-Zeichen
  - beginnen mit »%«
  - optionale Flags, Optionen und Längenangabe
  - abgeschlossen durch Umwandlungstyp

```c
printf("char: %c, string: %s, int: %i, hex: %x, pointer: %p",
       'c', "test", 123, 0x123, printf);
```

```
char: c, string: test, int: 123, hex: 123, pointer: 0x7fadd9056a64
```

cppreference – printf

# Ausgabefunktion `printf` (ii)

- wichtigste Typumwandlungen und Optionen

| Umwandlungstyp | Formatierung |
|---|---|
| %% | Prozentzeichen |
| %c | char |
| %s | Zeichenkette |
| %d / %i | dezimal int |
| %o %x | oktal hex int |
| %u | unsigned int |
| %f, %e, %g | double |
| %p | Zeiger *void |

| Option | Wirkung |
|---|---|
| - | Ausrichtung links |
| + | Vorzeichen immer |
| 0 | führenden Nullen |

```c
printf("%+d\n", 123);
printf("%+d", "hello"); // <- XXX: wrong type
```

```
+123
+1266475104
```

# Ausgabefunktion `printf` (iii)

- Längenangabe
  - minimale Länge
  - optionale Genauigkeit (».p«)

```c
double pi = 2*acos(0);
printf("%f\n", pi);
printf("%5f\n", pi);
printf("%5.2f\n", pi);
printf("%e\n", pi);
printf("%+010.5g\n", pi);
```

```
3.141593
3.141593
 3.14
3.141593e+00
+0003.1416
```

- Typ-Modifikatoren, z.B.
  - h → half
  - l → long
  - L → long double
  - z → size_t

```c
short s = 123;
long l = 123;
size_t n = 123;
printf("short: %hi, long: %li, size_t: %zu",
       s, l, n);
```

```
short: 123, long: 123, size_t: 123
```

# Ausgabefunktion `printf` (iv)

- Minimale Breite und Genauigkeit kann auch mit »`*`« angegeben werden
  - dann erwartet `printf()` jeweils einen zusätzlichen Parameter

```
for (int width = 10; width < 15; width += 4) {
    for (int prec = 0; prec < width; prec += 4) {
        printf("width %2d, prec %2d   ===>  ", width, prec);
        printf("'%*.*f'\n", width, prec, 12345.12345);
        //         ^ ^          |     |
        //         | '----------+-----'
        //         '------------'
    }
}
```

```
width 10, prec  0   ===>  '     12345'
width 10, prec  4   ===>  '12345.1234'
width 10, prec  8   ===>  '12345.12345000'
width 14, prec  0   ===>  '         12345'
width 14, prec  4   ===>  '    12345.1234'
width 14, prec  8   ===>  '12345.12345000'
width 14, prec 12   ===>  '12345.123449999999'
```

# Funktionszeiger

- Funktionsbezeichner sind Zeiger auf Beginn von Maschinencode

```c
int twice(int a) {
    return a * 2;
}

typedef int (*fp_t)(int);
fp_t also_twice = &twice;

int main() {
    SHOW(twice);
    SHOW(&twice);
    SHOW(also_twice);
    SHOW(&also_twice);
    SHOW(twice(2));
    SHOW(also_twice(2));
}
```

```
twice = 0x86ec8199
&twice = 0x86ec8199
also_twice = 0x86ec8199
&also_twice = 0x86ecb0e0
twice(2) = 4
also_twice(2) = 4
```

https://godbolt.org/z/6h5dahe96

```c
typedef struct { short x; short y; } point_t;
point_t* as_point(void *p) { return (point_t*)p; }

int cmp_x(const void* a, const void* b) {
    return as_point(a)->x - as_point(b)->x;
}
int cmp_y(const void* a, const void* b) {
    return as_point(a)->y - as_point(b)->y;
}
int main() {
    point_t points[] = {{0, 10}, {1, 20}, {2, 30}, {3, 10}};
    constexpr size_t N = sizeof(points)/sizeof(point_t);

    qsort(points, N, sizeof(point_t), cmp_x); // < compare by x
    SHOW_POINTS_N(points, N);

    qsort(points, N, sizeof(point_t), cmp_y); // < compare by y
    SHOW_POINTS_N(points, N);
}
```

```
points = {(0,10), (1,20), (2,30), (3,10)}
points = {(0,10), (3,10), (1,20), (2,30)}
```

https://en.cppreference.com/c/algorithm/qsort

# Undefiniertes Verhalten (*undefined behavior*)

- Warum gibt es im C-Standard undefiniertes Verhalten?
  - 64 Vorkommen von *“undefined behavior”*
  - § J.1 Unspecified Behavior: 63 Unterpunkte
  - § J.2 Undefined Behavior: 221 Unterpunkte
- erlaubt Compiler in bestimmten Situationen *irgendetwas*™ zu machen
  - ermöglicht damit bestimmte Optimierungen (in anderen Situationen), z.B.
    - keine Überprüfung ob Index innerhalb von Array
    - keine Überprüfung ob Zeiger gültig
    - keine Überprüfung auf Integer-Overflow
- als Programmierer immer vermeiden
  - z.B. mit Hilfe von Undefined Behavior Sanitizer

As-if rule - cppreference.com
Undefined behavior - cppreference.com
n3220